

Kirby 冒险之旅

基于 Qt6 和 C++17 的 2D 横版动作游戏

课程作业报告

组号：2 组

开发环境：Qt 6.11、MinGW 64-bit、C++17

开发工具：Qt Creator

一、项目概况

我们这次做的是一个 2D 横版动作冒险游戏，主角参考 Kirby 的玩法，可以移动、跳跃、翻滚、吞噬敌人，并在吞噬后获得不同能力。游戏有冒险关卡，也有单独的 Boss 模式，整体流程包括主菜单、选关、游戏过程、暂停设置、失败重来和结尾动画。

项目主要用 Qt 的 QGraphicsScene 和 QGraphicsView 来管理画面，用 QTimer 做每一帧的刷新。代码里把玩家、敌人、地图方块、弹幕、道具都写成了类，所以它不只是把素材放到窗口里，而是按 C++ 面向对象的方式组织出来的一个小型游戏项目。

我们已经完成的部分

1. 基础流程：主菜单、模式选择、暂停、设置、重新开始和结束动画。
2. 玩家操作：左右移动、跳跃、翻滚、飞行、攻击、吞噬和变身。
3. 地图系统：用 Tile 方块类搭建地图，包括草地、泥土、水面、水体、冰块、碎石块等。
4. 敌人与 Boss：普通小怪有不同能力，Boss 有血条、阶段变化和弹幕攻击。
5. 交互物：检查点、木箱、蛋糕、星星和终点等都能在关卡里正常工作。

二、功能设计

游戏开始后，玩家先进入主菜单，再选择冒险模式或 Boss 模式。冒险模式会从第一关开始，玩家在地图中前进，遇到敌人时可以攻击或吞噬；Boss 模式则直接进入 Boss 战。整个游戏的核心是让玩家在地图中移动并和不同对象发生交互。

1. 玩家部分

Player 类保存了角色当前状态，比如待机、行走、跳跃、翻滚、攻击、吞噬和消化。为了让状态不混乱，代码中使用了枚举 State 来区分这些动作。角色还有生命值、无敌时间、体力、朝向、当前能力、技能冷却等变量，每一帧都会根据按键和状态进行更新。

2. 地图部分

地图没有直接写死成图片，而是按方块来搭。每一种地形都是 Tile 类的一个对象，关卡加载时根据字符地图创建对应方块，再放到场景里。这样做的好处是碰撞、地形变化和关卡恢复都可以围绕 Tile 处理，后面改地图也比较方便。

3. 敌人部分

敌人统一从 Enemy 类派生。普通敌人用 MinionEnemy 表示，不同敌人携带不同 CopyAbility，玩家吞噬后可以获得火、冰、叶、电等能力。Boss 也是敌人的一种，不过血量更高，行为更复杂，有阶段切换和特殊弹幕。

4. 界面和音效

项目里还做了菜单按钮、血条、体力条、技能冷却提示、音效和背景音乐。虽然这些不是算法最复杂的部分，但它们决定了游戏玩起来是否完整。Qt 的资源文件把图片和音频集中管理，加载时直接使用资源路径。

三、程序结构

项目的主要逻辑集中在 `MainWindow` 中，它负责创建场景、读取关卡、保存对象列表，并在 `gameUpdate` 中统一调度。玩家、敌人、方块、弹幕和道具各自有单独的类文件，头文件写接口，`cpp` 文件写具体实现。这样的结构比较符合课程里讲的模块化写法。

1. `GameObject` 基类

`GameObject` 继承自 `QGraphicsPixmapItem`，并定义了 `vx`、`vy` 两个速度变量和 `updateLogic` 纯虚函数。`Player`、`Tile`、`Enemy`、`Projectile` 等类都可以看成 `GameObject` 的不同子类。这样写以后，游戏对象既能显示贴图，也能在每一帧更新自己的逻辑。

2. `Tile` 方块类

`Tile` 是地图系统的核心。它用 `TileType` 枚举区分草地、泥土、水面、水体、冰块、碎石块等类型，并用构造函数接收贴图。`MainWindow` 读取关卡时，会根据字符创建 `Tile`，再把固体方块放进 `floors`，把水相关方块放进 `waters`。角色和弹幕做碰撞时，就遍历这些列表。

3. `Player` 类

`Player` 里面变量比较多，因为角色的动作和能力都在这里维护。例如 `currentState` 表示当前动作，`currentForm` 表示当前复制能力，`cooldownTimer` 控制技能冷却，`hp` 和 `invulnTimer` 处理生命和受伤。动画帧也用 `QVector` 保存，状态改变时切换对应动画。

4. `Enemy`、`Boss` 和 `Projectile`

`Enemy` 类保存血量、伤害、是否死亡、能力类型等基础信息。`Boss` 在此基础上扩展出更复杂的行为，例如猪鲨 `Boss` 会冲刺、发射弹幕，并在某些阶段临时改变地形。`Projectile` 类用来表示玩家子弹、星星和 `Boss` 弹幕，每帧按速度移动，生命周期结束后删除。

四、几个关键实现

1. 关卡加载

loadLevel 函数会根据关卡编号清理上一关的对象，再创建新的地图、玩家、敌人和道具。地图部分用字符表示，例如某些字符对应草地方块，某些字符对应水块或冰块。代码根据行列位置计算坐标，再把对象加入 QGraphicsScene。

2. 碰撞处理

碰撞没有全部交给 Qt 自动处理，而是在游戏循环里按对象类型分别判断。角色和地面碰撞时，会先看矩形范围是否接近，再用 collidesWithItem 判断是否真的碰到。这样可以减少不必要的判断，也能分别处理站在地面、撞到墙、落入水中等情况。

3. 复制能力

玩家吞噬敌人后，会根据敌人的 CopyAbility 得到对应能力。火能力可以冲刺和爆炸，冰能力可以防御并冻结水面，叶能力可以释放叶子技能，电能力可以快速突进。每种能力都有独立冷却和动画，所以代码里用枚举和计时器把它们分开管理。

4. 检查点和恢复

检查点不仅保存玩家位置，还会保存一部分关卡状态。代码中用快照结构记录方块、敌人、道具和检查点信息，玩家失败后可以恢复到最近一次检查点。这个功能一开始容易出错，因为场景里的对象和容器里的指针必须一起更新，后来通过统一清理和重建对象解决。

5. Boss 阶段

Boss 不只是简单追踪玩家，而是有不同状态。例如准备、冲刺、冷却、阶段转换和死亡动画等。状态之间用枚举和计时器切换，每一帧根据当前状态执行不同逻辑。这样写虽然代码量多，但比把所有行为堆在一个 if 里清楚一些。

五、和 C++ 程序设计实习的关系

这个项目和 C++ 程序设计实习课程联系比较紧。很多功能都不是简单调用 Qt 控件完成的，而是需要自己设计类、维护对象、处理指针和容器。下面按课程知识点简单对应一下。

1. 类与对象：Player、Tile、Enemy、Projectile、Crate、Cake 等都是独立类，每个类负责自己的数据和行为。
2. 继承：Player、Tile、Enemy、Projectile 都继承 GameObject，Boss 类又继承敌人相关基类。
3. 多态：不同对象都实现 updateLogic，在主循环中可以按自己的规则更新。
4. 枚举：TileType、Player::State、CopyAbility、GameState 等枚举让状态更清楚，避免大量数字判断。
5. 构造函数：创建 Tile、Enemy、Projectile 时，会初始化贴图、速度、血量和能力等内容。
6. 容器：QList、QVector、QSet 用来保存方块、敌人、弹幕、动画帧和按键状态。
7. 指针和动态内存：场景中的对象很多都是 new 出来的，删除时要从 scene 和容器中同时移除。
8. 事件处理：键盘、鼠标和定时器事件共同组成游戏输入和刷新机制。

地图方块类是最典型的一部分

我们觉得地图系统最能体现课程内容。地图上的草地、水、冰块和碎石不是画在背景上的，而是一个个 Tile 对象。每个 Tile 都知道自己的类型，也能被放进不同列表里参与碰撞。比如玩家站在地面上时检查 floors，进入水里时检查 waters，Boss 改变地形时调用 Tile 的 changeType。这个过程把类、对象、枚举、构造函数、容器和指针都用到了。

六、测试和总结

1. 测试内容

1. 测试菜单、选关、暂停、设置和返回主菜单是否正常。
2. 测试玩家移动、跳跃、翻滚、吞噬、变身、受伤和死亡重开。
3. 测试地图方块碰撞、水面交互、冰块转换、木箱推动和检查点恢复。
4. 测试普通敌人的巡逻、攻击、被吞噬和能力获取。
5. 测试 Boss 血条、阶段变化、弹幕、死亡动画和奖励掉落。

2. 遇到的问题

开发时比较麻烦的是对象清理。弹幕、敌人、方块和道具都在场景里，同时又保存在不同 QList 中。如果删除顺序不对，就可能出现空指针或画面残留。后来我们尽量在删除对象时同时处理 scene 和容器，并且在循环删除时使用倒序遍历。

另一个问题是玩家状态太多。比如攻击、吞噬、翻滚、变身和受伤之间可能互相影响，如果只靠几个 bool 变量会很乱。后来把主要动作放到 State 枚举里，再用冷却计时器控制技能，逻辑会清楚很多。

3. 总结

这次作业让我们把 C++ 课程里的很多知识真正用到一个能运行的程序里。游戏最后还有可以继续优化的地方，比如代码可以再拆分得更细，关卡编辑也可以做得更方便。但从课程实践角度看，我们已经完成了一个包含地图、玩家、敌人、Boss、道具、音效和界面的完整小游戏。